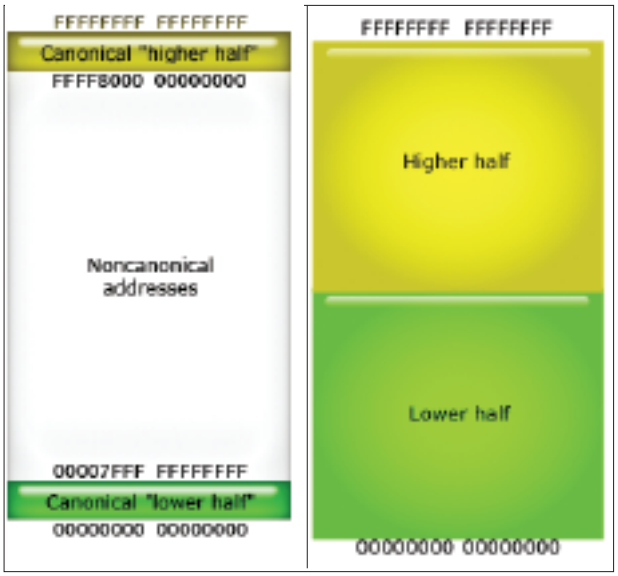
A Peek Into x86_64

If 64-bit computing has been around since the 1960s (it started with the IBM Stretch Supercomputer in 1961), what took it so long to get to our desktops? One word: compatibility. By the time 64-bit computing was considered for computers less glamorous than the world's fastest supercomputers, a sizeable chunk of the computing world had already chosen to go the way of the Intel 8086 and its successors—the x86 series. Programs were written in its native 'instruction set', which could only be understood by these processors and no other. To ask the world to move to 64-bit processors and leave all their precious x86 applications behind would be a laughable idea—almost as silly as asking all software vendors to rewrite their programs to use the instruction set of a new processor.

So in 1999, AMD published a way out—they took the existing x86 processor architecture and modified it to support 64-bit instructions, while still retaining the ability to run old, 32-bit x86 instructions—the "x86-64" concept materialised with the first Athlon 64 in 2003. It meant that we could now enjoy the goodness of 64-bit programs, but didn't have to sacrifice our old 32-bit applications to be able to do so.

The new processors' ALU works with 64-bit numbers; existing registers are now 64 bits wide, and more registers have been added to hold temporary data. This is a good thing—the data in registers can be accessed the fastest, and to have more registers is to have more data close to the processor. This, in turn, means that the processor can spend more time processing, and less time waiting for data to be brought to it from the system's memory. It also means that data that needs to be immediately available need not be sent even to the cache—another boost to performance.



The x86-64 design uses 48 bits to address memory. The section at the top is used by the operating system and devices, and the section at the bottom is called "user memory"—for your programs. The possible locations in between won't be used for now. The second image shows an estimate of what 64-bit addressing would be like under Windows. Windows NT uses the top half of all memory addresses for itself and devices, and the bottom half can be given to programs. This is why Windows XP will only give 2 GB of memory to programs even if you've got 4 GB of RAM

The x86-64 architecture, however, doesn't use all 64 bits for memory addresses—it uses 48, making the total addressable memory 256 terabytes. Before you start yelling that you've been cheated of the remaining millions of terabytes, consider that 256 terabytes is a *lot* of memory, and ought to be enough for anybody—for now, at least. Translating a 64-bit integer into a physical memory address is work on the part of the processor, and it's simply not worth it—it'll be a really long while before 18 million terabytes of memory becomes a realistic figure, so why waste the processor's time translating 64-bit addresses? As for the day when it does become a realistic number, there'll be a processor that'll use all the 64 bits—x86-64's future-proof design makes room for that.

All said and done, what do *you* get out of all this? And do you need more than 4 GB to "enjoy" 64-bits of computing goodness?

The Real Advantage (?)

You have no choice but to buy a 64-bit processor today, but must you use Windows XP x64 Edition (or 64-bit Linux) as your new OS? Without 4 GB of RAM, you obviously can't take advantage of the ability to load plenty of data into your system's memory. You can, however, take advantage of faster calculations—applications like audio and video editors will run a bit faster, but not too much, unless you give them more memory to play with. Adobe Photoshop, while not available in a 64-bit version, will use system memory over the 4 GB mark as a scratch disk if it's running on a 64-bit processor with a 64-bit OS. If you're not a professional in any of these fields, however, this will matter little.

Must You Care?

Did you care that when you ditched Windows 98 for XP, you had finally left 16-bit computing behind and ascended into the world of true 32-bit computing? We thought not. Software has traditionally taken a long while to come to terms with hardware, and the day when all code is 64-bit is still a good way off.

For those of us who aren't audio, video, math or imaging professionals, the transition to 64-bitness will be as smooth as it was from 16- to 32-bit. And as long as our applications still work when we get there, it'll be painless. ■

nimish.chandiramani@thinkdigit.com